

O objetivo desse projeto é ampliar o conhecimento sobre os sistemas operacionais através da implementação da modificação no kernel do sistema operacional didático XV6.

O nosso grupo escolheu implementar uma chamada de sistema com o objetivo de obter o turn around time de um processo e analisar o impacto da variação do quantum no algoritmo de escalonamento Round Robin.

O turn around time é o tempo total de existência de um processo, ou seja, o tempo exato em que ele foi alocado na memória até o momento em que ele é desalocado. Esses tempos foram obtidos modificando a struct do processo, incluindo nele o momento em que ele foi criado e o momento em que ele foi finalizado.

Já o round robin é um algoritmo de escalonamento no qual cada processo recebe uma fatia do tempo da CPU, ou seja, cada processo passa uma determinada quantidade de tempo antes de ser retirado de lá para que o próximo processo tenha acesso à CPU. O tempo dado ao processo na CPU é chamado de quantum.

Como o XV6 implementa o round robin com quantum de 1 passagem pelo usertrap, precisamos alterar algumas coisas para que possamos variar o quantum. Dentro do struct proc, foi necessário colocar um contador de quantas vezes ele passava pelo usertrap (tempo_quantum). Para definir o quantum, adicionamos uma constante QUANTUM no trap.c. Toda vez que a CPU recebe uma interrupção por tempo, ele verifica se o tempo_quantum é maior ou igual ao tempo definido na constante, se for, ele retira o processo da CPU.

Dentro do usertrap() as alterações ficaram da seguinte maneira:

```
//mudar o quantum aqui!
#define QUANTUM 5

//codigo original no meio

if (which_dev == 2){
    struct proc *p = myproc();
    if (p && p->state == RUNNING) { //verificando se o processo
existe e esta rodando
        p->tempo_quantum++;

        if (p->tempo_quantum >= QUANTUM) { //autoexplicativo
            p->tempo_quantum = 0;
            yield();
        }
    }
}
```

Já as alterações no kexit() ficaram assim:

```
//depois de liberar os arquivos pega o tempo de finalizacao do processo
p->final = ticks;
```

Alterações no allocproc():

```
//coloca o tempo de criacao do processo
p->criacao = ticks;
p->final = 0;
p->tempo_quantum = 0;
```

Alterações no struct proc:

```
//adicionei o tempo de criacao e de fim
uint64 criacao;
uint64 final;

uint64 tempo_quantum;
```

Adicionando a chamada de sistema no syscall.c:

```
static uint64 (*syscalls[])(void) {
//chamadas originais do sistema
[SYS_tatime] sys_tatime,
```

```
extern uint64 sys_tatime(void);
```

Adicionando a chamada de sistema no syscall.h:

```
#define SYS_tatime 23
```

Adicionando a chamada dentro de user.h:

```
int tatime(void);
```

Adicionando a chamada dentro de usys.pl:

```
entry("tatime");
```